

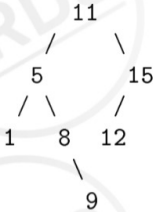


Nombre:

Grupo:

Pregunta 1:

- [ ] Para simular una cola circular dinámica (p.e. procesos de un S.O) en el que los datos entran siempre por el mismo lugar pero salen en cualquier momento es igualmente eficiente utilizar una lista (list) que un vector de STL.
- [ ] Se ha pensado en implementar una matriz para almacenar todos los jugadores convocados en todos los partidos de todas las jornadas de la liga de fútbol. Esta sería una posible estructura de datos para crear dicha tabla: vector<vector<vector <Jugador> > > jugadores.
- [ ] Un árbol binario de altura 3 (4 niveles) y 8 nodos puede ser completo.
- [ ] Dado el siguiente árbol AVL, la inserción del 13 requiere una rotación doble tipo 3.



- [ ] Una lista doblemente enlazada permite realizar búsquedas

- binarias en tiempo  $O(\log n)$  si los datos se encuentran ordenados.
- [ ] Es posible que sea necesario seguir el proceso de búsqueda en una tabla hash con dispersión cerrada y cubetas de tamaño 5 cuando se llega a una cubeta con 3 datos.
- [ ] En un par de horas se han contabilizado 20 búsquedas y 3 inserciones en un sistema de ficheros con un índice simple y dos índices secundarios, sin embargo el fichero de datos no ha modificado su tamaño, lo que resulta imposible.
- [ ] Es posible que una casilla de una malla regular esté vacía, al igual que un nodo de un quadtree represente un área vacía.
- [ ] La función de djb2 de Dan Bernstein es una función de dispersión utilizada normalmente para cadena de caracteres
- [ ] Si se implementa el padrón electoral de todos los ciudadanos de cara a las elecciones mediante un conjunto disjunto del tipo <municipio, dni>, siendo municipio un entero de 0 a num-Municipios y dni un valor alfanumérico, entonces la operación para saber si hay duplicidades, es decir, si una persona ya está empadronada en otro municipio puede resolverse en tiempo constante.

**Pregunta 2:** Definid una función de dispersión doble con  $h1(x)=x \% 11$  y  $h2(x)=x \% 7$  para una tabla con dispersión cerrada de tamaño igual a 11 en el que se inserten/borren los siguientes datos enteros:  
I(3), I(24), I(13), I(10), I(40), B(13), I(2), I(6), I(9), B(24), I(25)

Pregunta 3:

1. Implementar las funciones:  
T Avl<T>::minimo();  
T Avl<T>::maximo();  
que devuelven respectivamente el menor y el mayor de los datos de un árbol AVL
2. Implementar la siguiente función que elimina todas las instancias repetidas de una lista simplemente enlazada y circular (no se deben utilizar iteradores):  
ListaEnlazadaC<T>::unique();

**Pregunta 4:** Una planta de una gran fábrica ha implementado un enorme sistema de paso de envíos de unos puestos de operación a otros. Consiste en una gran cadena circular con más de 200 cestas que recorren todos los departamentos de dicha fábrica. El circuito se mueve automáticamente a intervalos de tiempo, parándose en cada puesto el tiempo suficiente para que el personal ponga o recoja un envío. El circuito sólo funciona en una dirección, lo que significa que si se quiere enviar del puesto  $i$  al  $i-1$ , tardará en llegar casi un ciclo entero.

Las cestas están perfectamente estructuradas y pueden recibir envíos de tamaño uno, dos o cuatro. Cuando un puesto solicita un envío debe especificar el destino y el tamaño. La gran cadena se mueve y si llega a un puesto que tiene una petición de recogida y la cesta tiene espacio suficiente, entonces pondrá una luz verde al pararse indicando al operario del puesto que podrá colocar el envío sin problemas en la bandeja. El proceso para llevar mercancía es similar, cuando una cesta llega al puesto destino, se enciende una luz roja al hacer la parada. Esto indica al operario que le está llegando mercancía y éste deberá retirarla.

El funcionamiento de cada clase es la siguiente:



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional

**Puesto:** Un puesto tiene un número que lo identifica de 0 hasta *CadenaSuministros::numPuestos*. Este número puede cambiar de una configuración a otra de la fábrica. Cuando un puesto quiere enviar un paquete debe realizar una *nuevaPeticiónEnvio()*, indicando el destino y el tamaño de la carga (1, 2, o 4). Estas peticiones se almacenan en una cola particular de *PeticiónEnvio*, y se irán atendiendo por estricto orden de llegada. Cuando una cesta tenga espacio para hacerse cargo de la siguiente petición, la cesta se pondrá en color verde indicando que se podrá depositar la carga en dicha cesta. Entonces se llamará a *recogePetición()* y se eliminará dicha petición de la cola. La función *verSiguientePetición()* devuelve la siguiente petición, de esta forma se sabe si la petición puede ser atendida por la cesta que en ese momento está accesible. Aunque una cesta pueda admitir más de un pedido del mismo puesto, sólo se atiende uno por cada parada de la cadena.

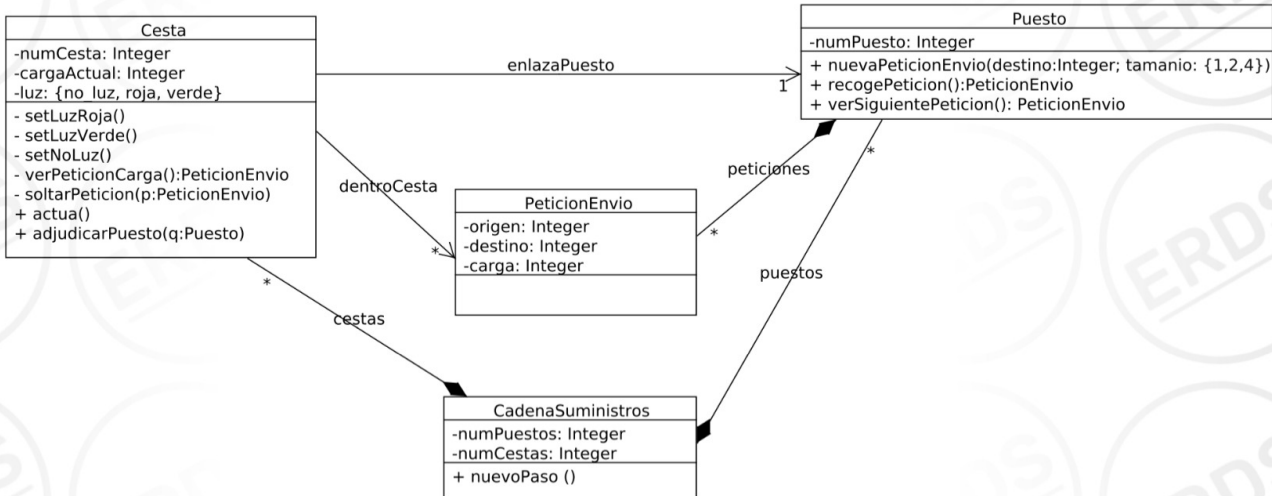
Cuando una cesta se sitúa en el puesto de destino de alguno de sus paquetes, la luz será roja, y el paquete será depositado en su destino, desapareciendo del sistema.

**PeticionEnvio:** Se crea por la clase *Puesto* y se elimina por la cesta cuando el paquete que contiene llega a su destino.

**Cesta:** Representa una cesta de la muchas que tiene la cadena de suministros y se identifica por un número entero. Tiene cuatro compartimentos y admite bandejas de tamaño uno, dos o cuatro. De este modo si una cesta tiene una carga actual de 2 y le hacen una petición de tamaño 2 podrá aceptarla, si le piden de tamaño 4 no podrá hacerlo, así que pasará de largo y no encenderá la luz verde. Tendrá tres estados cada vez que se para en un puesto, verde para aceptar carga previa comprobación de espacio, roja para indicar que deja toda o parte de su carga en dicho puesto o apagada. La función *actua()* es la que se ejecuta cada vez que se para en un puesto: lo primero que hace es apagar la luz y *verPeticionCarga()* del puesto actual para saber si dicho puesto tiene peticiones y por tanto si las puede atender. En caso afirmativo pone su estado a verde y se supone que le añaden la nueva carga. No hace más comprobaciones de error. La función *actua()* también comprueba si el puesto actual es destino de alguno de sus paquetes, en tal caso lo suelta con *soltarPetición()*, eliminándolo definitivamente del sistema. Es posible que suelte y recoja mercancía del mismo puesto, en tal caso primero se pone la luz roja para soltar y luego la verde para recoger.

La función *adjudicarPuesto()* sirve para enganchar en casa paso una cesta con el siguiente puesto.

**CadenaSuministros:** Es la clase que controla todo el proceso, se encarga en cada paso de la iteracion de enlazar cada cesta con el puesto siguiente. En cada iteración adjudica un nuevo puesto a cada cesta con *Cesta::adjudicaPuesto()* y luego le indica que actúe con *Cesta::actua()*.



Esta obra pertenece a ERDS bajo la licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional